

Forest Intelligence

What I built

Forest Intelligence is a Streamlit app that estimates individual tree crowns from RGB forest imagery. Someone can start immediately with a bundled NEON example scene, or bring their own JPG, PNG, or GeoTIFF plus an optional KML boundary. The app detects crowns, reports a count and canopy coverage, and - this is the part I cared about most - is upfront about how much to trust the result and which parts of it need a second look. It also exports the tree-level data as a CSV, an annotated image, and a short PDF report.

I built the interface for someone who has never seen it before: a preview before anything runs, a real progress checklist during analysis instead of a spinner, plain-language explanations next to every number, and technical detail one click away rather than in the way.

Architecture

An uploaded image goes through rasterio (falling back to Pillow if rasterio can't read it) to check for a coordinate reference system and a ground sample distance. A separate parser reads KML polygons and checks whether the boundary actually overlaps the image. DeepForest then runs pretrained RetinaNet inference - single-pass for small images, tiled for larger ones - at a confidence floor of 0.30. Everything after that point is a plain measurement layer: box areas, coverage percentage, confidence/overlap/shadow flags, a reliability score, size distribution, and the scenario-slider arithmetic. That layer never touches the model directly, which is what let me test the math against synthetic detections instead of needing a live network connection every time I wanted to check it. Streamlit renders the result on top.

There's no database, no login, and no LLM call anywhere in the request path. The handful of contextual facts shown next to the results come from a small JSON file I wrote and sourced by hand, not a live model call.

Key decisions

I used DeepForest rather than training something from scratch - it's peer-reviewed, MIT-licensed, ships pretrained weights and an example scene, and a weekend isn't enough time to train anything that would beat it. DeepForest returns boxes, not crown-shaped outlines, so "canopy area" in this app means the area of a box drawn around a crown, which tends to run a little high on irregular crowns. I say that plainly in the app instead of implying a precision the model doesn't have. Along the same lines, an image with no coordinate reference system has no way to establish ground scale, so the app reports pixel counts and says so rather than guessing at square metres.

Reliability is computed, not asserted. It comes from actual confidence scores, how much boxes overlap, a brightness check standing in for shadow detection, whether the image is georeferenced, and how many trees were even found - the underlying counts are shown, not hidden behind a single score. And I left out anything about carbon, biomass, or species: a crown count alone doesn't get you there, and guessing would have been exactly the kind of invented precision this brief is designed to catch.

What worked

Everything that doesn't depend on the model runs end to end: loading images with and without spatial reference, KML parsing and overlap checks, the area and coverage math, the review flags, the reliability score, the size distribution, the facts panel, and all three exports. The measurement layer has unit tests covering empty results, clean detections, the physical-unit conversion, overlap and low-confidence flagging, and the scenario slider - they run against synthetic detections, so they pass without needing the model or any real imagery. The example-scene path is meant to take a stranger from a blank screen to a full result with nothing to configure.

Limitations

- **Boxy estimates:** It draws square boxes around trees instead of exact outlines, so the area estimate is slightly larger than the real tree shape.
- **No carbon or health data:** It only counts trees and measures area. It cannot figure out carbon amounts, tree species, age, or health.
- **Needs location data:** To measure in square meters, the uploaded image must have built-in spatial data. Otherwise, it only gives pixel percentages.
- **Struggles with crowds and shadows:** It has a hard time separating trees that overlap heavily or are covered by dark shadows.
- **Plays favorites:** It was built using specific types of forest images, so it might not work as well on forests or camera angles that look very different.
- **Slow wake-up:** Because it runs on a free hosting plan, the live website might take 30 to 60 seconds to load if no one has used it recently.

Technologies

Python, Streamlit, DeepForest (PyTorch/RetinaNet), rasterio, Shapely, pyproj, Pillow, NumPy, pandas, ReportLab/fpdf2, pytest. I used Claude for a good share of the coding - scaffolding, the geospatial math, the test suite - which is the kind of assistance this brief said was welcome. The numbers the app reports come from that code and the pretrained detector, not from anything a language model asserted on its own.